

Trabalho Prático – Etapa 3 – GCC130

Análise Semântica e Geração de Código Intermediário

Caio Bueno Finocchio Martins
Lana da Silva Miranda
Nina Tobias Novikoff da Cunha Ribeiro

Universidade Federal de Lavras (UFLA)
Professor: Ricardo Terra

1 Introdução

A terceira etapa do trabalho prático teve como objetivo complementar o compilador desenvolvido nas fases anteriores, adicionando a análise semântica e a geração de código intermediário para a minilinguagem didática. Enquanto a etapa léxica ficou responsável pelo reconhecimento dos tokens e a etapa sintática pela validação estrutural do programa, esta etapa passou a verificar o significado das construções reconhecidas e a produzir uma representação intermediária do código-fonte.

A análise semântica implementada verifica aspectos que não podem ser garantidos apenas pela gramática, como o uso de identificadores declarados, a compatibilidade de tipos, a redeclaração no mesmo escopo, o uso correto de funções, a validação de argumentos e a consistência dos comandos de retorno. Para isso, foi utilizada uma tabela de símbolos com suporte a escopos aninhados, permitindo que o compilador diferencie variáveis globais, variáveis locais, parâmetros e funções.

Além da análise semântica, foi implementada a geração de código intermediário em formato de código de três endereços. Essa representação utiliza temporários, como `t1`, `t2` e `t3`, e rótulos, como `L1`, `L2` e `L3`, para representar expressões, atribuições, comandos de entrada e saída, estruturas condicionais, laços, chamadas de função e retornos. A escolha por uma representação textual simples permitiu manter a implementação didática e compatível com o escopo do trabalho.

2 Análise Semântica

2.1 Estrutura da Tabela de Símbolos

A tabela de símbolos foi implementada a partir de uma estrutura encadeada de ambientes (ou escopos), representada pela estrutura `Env`. Cada escopo possui uma lista de símbolos (que representa a tabela de símbolos daquele escopo), o nível do escopo atual e

um ponteiro para o escopo anterior. Dessa forma, cada bloco do programa pode possuir sua própria tabela, mantendo uma ligação com os escopos externos.

Cada símbolo armazenado contém informações essenciais para a análise semântica, como o lexema, o tipo, a categoria, o nível de escopo e, no caso de funções, a lista de parâmetros. A estrutura utilizada para representar um símbolo segue a seguinte ideia:

```
1 typedef struct Simbolo {
2     int id;
3     char lexema[100];
4     int ocorrencias;
5     Ttype tipo;
6     enum CategoriaId categoria;
7     ParametroFuncao *parametrosFuncao;
8     struct Simbolo *prox;
9     int nivelEnv;
10 } Simbolo;
```

Listagem 1: Estrutura simplificada de símbolo

A categoria do identificador foi usada para diferenciar variáveis, funções e parâmetros. Essa separação foi necessária porque um mesmo lexema pode estar semanticamente correto em um contexto e incorreto em outro. Por exemplo, uma função pode ser chamada, mas não pode ser usada diretamente como variável no lado esquerdo de uma atribuição ou dentro de um comando `read`.

2.2 Gerenciamento de Escopos

O gerenciamento de escopos foi implementado por meio das funções `criarEnv` e `fecharEnv`. Sempre que o parser entra em um novo bloco delimitado por chaves, um novo ambiente é criado e passa a ser o ambiente atual. Ao sair do bloco, o ambiente atual é fechado e o compilador retorna ao ambiente anterior.

Essa organização permite representar escopos aninhados de forma simples. A busca por identificadores é feita primeiro no escopo atual e, caso o símbolo não seja encontrado, continua nos escopos externos. Com isso, a linguagem permite o uso de identificadores declarados em blocos mais externos, mas impede a redeclaração de um mesmo identificador dentro do mesmo escopo.

A função `buscarSimboloEnv` verifica apenas o ambiente atual, sendo usada para detectar redeclarações. Já a função `buscarSimboloGeral` percorre a cadeia de ambientes, sendo usada para verificar usos de identificadores já declarados.

```
1 Simbolo* buscarSimboloGeral(char* lexema){ //Procura em todos os envs
```

```

2   Env *env = envAtual;
3   Simbolo *encontrado;
4
5   while(env!=NULL){
6       if (env != NULL){
7           encontrado = buscarSimboloEnv(lexema, env);
8           if (encontrado != NULL){
9               return encontrado;
10              } else {
11                  env = env->prev;
12              }
13          }
14      }
15      return NULL;
16  }

```

Listagem 2: Ideia geral da busca em escopos aninhados

No caso das funções, o gerenciamento de escopo ocorre em duas etapas. Primeiro, o identificador da função é inserido no escopo em que a declaração aparece, normalmente o escopo global, permitindo que ela seja encontrada posteriormente em chamadas. Em seguida, ao iniciar o corpo da função, é criado um novo ambiente para representar o escopo local da função. Nesse ambiente são inseridos os parâmetros formais e as variáveis declaradas dentro do corpo. Durante a análise desse corpo, o compilador mantém o contexto da função em declaração por meio de `funcaoDeclaracaoAtual` e `tipoRetornoAtual`, o que permite validar comandos `return` de acordo com o tipo da função. Ao final da função, o ambiente local é fechado, removendo os parâmetros e variáveis locais da visibilidade, enquanto o símbolo da função permanece disponível no escopo onde foi declarado.

2.3 Tabela de Regras de Tipagem

A tabela a seguir apresenta as principais regras de tipagem adotadas na análise semântica. Durante a implementação, cada expressão carrega seu tipo e o endereço usado na IR, permitindo verificar compatibilidade de tipos e gerar o código intermediário correspondente.

Construção	Regra Semântica	Resultado
Declaração de variável	O identificador não pode estar declarado no mesmo escopo.	Símbolo inserido na tabela.

Declaração com inicialização	O tipo da expressão deve ser compatível com o tipo declarado. É permitida promoção de <code>int</code> para <code>float</code> .	Inicialização aceita.
Atribuição	O identificador deve existir e não pode ser função. O tipo da expressão deve ser compatível com o tipo da variável.	Atribuição aceita.
Soma, subtração, multiplicação e divisão	Os operandos devem ser numéricos. Se algum operando for <code>float</code> , o resultado também será <code>float</code> .	<code>int</code> ou <code>float</code> .
Módulo	Ambos os operandos devem ser do tipo <code>int</code> .	<code>int</code> .
Operadores relacionais	Os operandos devem ser numéricos.	<code>int</code> , representando falso ou verdadeiro.
Operadores lógicos	Os operandos devem ser numéricos.	<code>int</code> , representando falso ou verdadeiro.
Negação lógica	O operando deve ser numérico.	<code>int</code> , representando falso ou verdadeiro.
Menos unário	O operando deve ser <code>int</code> ou <code>float</code> .	Mesmo tipo do operando.
Condição de <code>if</code>	A condição deve ser uma expressão numérica.	Condição aceita.
Condição de <code>while</code>	A condição deve ser uma expressão numérica.	Condição aceita.
Comando <code>read</code>	O identificador deve existir e não pode ser função.	Entrada aceita.
Chamada de função	O identificador deve existir e ser função. A quantidade e os tipos dos argumentos devem ser compatíveis com os parâmetros.	Tipo de retorno da função.
Comando <code>return</code>	Deve ocorrer dentro de uma função e o tipo retornado deve ser compatível com o tipo declarado da função.	Retorno aceito.

Tabela 1: Principais regras de tipagem adotadas na análise semântica

3 Geração de Código Intermediário

3.1 Estrutura Geral da IR

A geração de código intermediário foi implementada em formato textual, seguindo o modelo de código de três endereços. As instruções são emitidas pela função `gen` e armazenadas em um buffer textual, dispensando o uso de uma estrutura separada de quádruplas, por decisão de simplicidade. Para representar resultados intermediários de expressões, o compilador utiliza temporários gerados por `newTemp`, como `t1`, `t2`, `t3`. Já para representar desvios de controle, são usados rótulos gerados por `newLabel`, como `L1` e `L2`. Cada expressão carrega no atributo `addr` o local onde seu resultado está armazenado, que pode ser uma variável, uma constante ou um temporário. Essa estrutura permite que comandos condicionais e laços sejam traduzidos por meio de instruções como `ifFalse`, `goto` e rótulos, usadas na geração de IR para `if/else` e `while`.

3.2 Geração de IR para `if` e `if/else`

A geração de IR para comandos condicionais utiliza o comando `ifFalse`. No caso de um `if` simples, o compilador gera um rótulo para o ponto imediatamente após o corpo do comando. Caso a condição seja falsa, o fluxo salta para esse rótulo.

Para o código:

```
1 if ($a < $b)
2 $c = 1;
```

Listagem 3: Exemplo de comando `if` simples

a IR gerada segue a forma:

```
1 t1 = $a < $b
2 ifFalse t1 goto L1
3 $c = 1
4 L1:
```

Listagem 4: IR gerada para `if` simples

No caso do `if/else`, são usados dois rótulos: um para o início do bloco `else` e outro para o fim da estrutura. Após executar o corpo do `if`, é gerado um `goto` para impedir que o bloco `else` seja executado indevidamente.

```
1 t1 = $a > $b
2 ifFalse t1 goto L1
3 $c = 1
```

```

4 goto L2
5 L1:
6 $c = 2
7 L2:

```

Listagem 5: IR gerada para if/else

Para permitir que o `ifFalse` fosse emitido antes do corpo do `if`, foi criada a regra auxiliar `marca_if`. Essa regra não altera o significado da gramática; ela funciona apenas como um ponto intermediário de ação semântica para gerar o desvio condicional no momento correto.

3.3 Geração de IR para while

A estrutura `while` utiliza dois rótulos principais: um para o início do laço e outro para a saída. Primeiro, o compilador gera o rótulo de início. Em seguida, avalia a condição. Caso a condição seja falsa, o fluxo salta para o rótulo de saída. Ao final do corpo do laço, é gerado um `goto` de volta para o início.

Para o código:

```

1 while ($x < 5) {
2   $x = $x + 1;
3   $z = $z + $x;
4 }

```

Listagem 6: Exemplo de comando while

a IR gerada segue a forma:

```

1 L1:
2 t1 = $x < 5
3 ifFalse t1 goto L2
4 t2 = $x + 1
5 $x = t2
6 goto L1
7 L2:

```

Listagem 7: IR gerada para while

Assim como no `if`, foi criada uma regra auxiliar chamada `marca_while`. Essa regra permite emitir o rótulo de início antes da análise da condição e do corpo do laço, mantendo a implementação simples e compatível com o funcionamento do Bison.

4 Tradução Dirigida por Sintaxe

A implementação utilizou Tradução Dirigida por Sintaxe para associar ações semânticas às produções da gramática. Seguindo esse modelo, os atributos das expressões carregam informações como tipo e endereço na IR. O atributo `type` representa o tipo da expressão, enquanto `addr` representa o local em que o resultado está armazenado, podendo ser uma variável, constante ou temporário.

A TDS abaixo resume a tradução adotada para atribuições e expressões aritméticas.

Produção	Regras Semânticas
$S \rightarrow id = E ;$	<pre>s = buscar(id.lexema) verificarCompatibilidade(s.type, E.type) a = widen(E.addr, E.type, s.type) gen(id.lexema = a)</pre>
$E \rightarrow E1 + T$	<pre>E.type = tipoResultante(E1.type, T.type) a1 = widen(E1.addr, E1.type, E.type) a2 = widen(T.addr, T.type, E.type) E.addr = newTemp() gen(E.addr = a1 + a2)</pre>
$E \rightarrow E1 - T$	<pre>E.type = tipoResultante(E1.type, T.type) a1 = widen(E1.addr, E1.type, E.type) a2 = widen(T.addr, T.type, E.type) E.addr = newTemp() gen(E.addr = a1 - a2)</pre>
$T \rightarrow T1 * F$	<pre>T.type = tipoResultante(T1.type, F.type) a1 = widen(T1.addr, T1.type, T.type) a2 = widen(F.addr, F.type, T.type) T.addr = newTemp() gen(T.addr = a1 * a2)</pre>
$T \rightarrow T1 / F$	<pre>T.type = tipoResultante(T1.type, F.type) a1 = widen(T1.addr, T1.type, T.type) a2 = widen(F.addr, F.type, T.type) T.addr = newTemp() gen(T.addr = a1 / a2)</pre>
$T \rightarrow T1 \% F$	<pre>verificarInteiros(T1.type, F.type) T.type = int T.addr = newTemp() gen(T.addr = T1.addr \% F.addr)</pre>

F -> - F1	verificarNumerico(F1.type) F.type = F1.type F.addr = newTemp() gen(F.addr = - F1.addr)
F -> id	s = buscar(id.lexema) F.type = s.type F.addr = id.lexema
F -> numero	F.type = numero.type F.addr = numero.lexema

Tabela 2: TDS para atribuições e expressões aritméticas

Também foi usada uma TDS para a geração de código intermediário em comandos de controle de fluxo. Nesses casos, os atributos principais são rótulos gerados com `newLabel()`, usados para representar desvios condicionais e incondicionais.

Produção	Regras Semânticas
M_if -> if (E)	verificarNumerico(E.type) M_if.false = newLabel() gen(ifFalse E.addr goto M_if.false)
S -> M_if S1	gen(M_if.false:)
S -> M_if S1 else S2	S.next = newLabel() gen(goto S.next) gen(M_if.false:) S2.code gen(S.next:)
M_while -> while (	M_while.begin = newLabel() gen(M_while.begin:)
S -> M_while E) S1	verificarNumerico(E.type) S.next = newLabel() gen(ifFalse E.addr goto S.next) S1.code gen(goto M_while.begin) gen(S.next:)

Tabela 3: TDS para comandos de controle de fluxo

5 Decisões de Projeto

As seguintes decisões de projeto foram tomadas durante a implementação da análise semântica e da geração de código intermediário.

1. **Uso de ambientes encadeados para escopos.** Decidiu-se representar os escopos por meio da estrutura `Env`, encadeando cada novo ambiente ao ambiente anterior a partir do atributo `prev`. Essa decisão simplificou a busca por identificadores em escopos aninhados e permitiu detectar redeclarações apenas no escopo atual.
2. **Uso de categorias para identificadores.** Os símbolos foram classificados como variável, função ou parâmetro. Essa decisão permitiu verificar usos incorretos, como tentar atribuir valor a uma função ou usar uma função dentro de `read`.
3. **Uso de variáveis globais de contexto no Bison.** Foram utilizadas variáveis globais como `envAtual`, `tipoAtual`, `funcaoDeclaracaoAtual`, `funcaoChamadaAtual` e `tipoRetornoAtual`. Essa decisão foi adotada por simplicidade, pois ações semânticas em Bison frequentemente precisam compartilhar estado entre diferentes pontos da gramática.
4. **Separação entre função em declaração e função em chamada.** Inicialmente, uma única variável de contexto poderia causar confusão entre a função cujo corpo estava sendo analisado e a função chamada em uma expressão. Por isso, decidiu-se separar `funcaoDeclaracaoAtual` e `funcaoChamadaAtual`. Essa separação evita que uma chamada feita dentro de uma função interfira na validação de validação de `return` ou na lista de parâmetros da função declarada.
5. **Uso de contexto válido e inválido para chamadas de função.** Durante a validação de chamadas, foi necessário tratar também casos em que o identificador chamado não existe ou não representa uma função. Para evitar acessos indevidos a ponteiros nulos e, ao mesmo tempo, permitir que a análise continue reportando outros erros, foi adotado um campo inteiro de validade na struct de função, recebido pela função `empilharFuncao` como parâmetro. Assim, um contexto pode ser marcado como válido ou inválido. Quando o contexto é inválido, a validação de parâmetros e argumentos é ignorada, mas a estrutura da análise permanece consistente. Essa necessidade surgiu porque, ao final da leitura dos argumentos da função chamada, o contexto dessa chamada é removido da pilha de funções. Dessa forma, se uma chamada inválida não fosse empilhada, a remoção feita no final iria acabar retirando incorretamente a função anterior da pilha. Por isso, até as chamadas inválidas são empilhadas como contextos inválidos, apenas para manter o controle correto da pilha e evitar que um erro em uma chamada afete a análise das próximas.

6. **Uso de variável `parametroAtual` para percorrer a lista de parâmetros.** Durante a validação de chamadas de função, foi utilizado o campo `parametroAtual` para percorrer a lista de parâmetros formais da função chamada. Conforme cada argumento é lido pela gramática, ele é comparado com o parâmetro correspondente, verificando a compatibilidade de tipos e avançando para o próximo parâmetro esperado. Caso sejam lidos mais argumentos do que o esperado, é reportado erro de quantidade excessiva; ao final da chamada, se ainda restarem parâmetros não verificados, é reportado erro de quantidade insuficiente.
7. **Expansão de `Ttype` com o campo `addr`.** A estrutura de tipo passou a armazenar também o endereço da expressão na IR. Essa decisão permitiu reaproveitar os atributos já usados na análise semântica para transportar também o nome da variável, constante ou temporário gerado.
8. **Uso de código de três endereços textual.** A IR foi implementada como uma sequência textual de instruções, em vez de uma estrutura complexa de objetos ou nós. Essa decisão manteve a implementação simples e adequada ao escopo do trabalho.
9. **Não implementação de DAG, otimizações, blocos básicos ou código objeto.** Esses elementos aparecem em fases posteriores de um compilador, mas foram deixados fora do projeto por não serem necessários para a entrega. A decisão foi manter o foco em análise semântica e geração de IR.
10. **Uso de valores inteiros para expressões lógicas.** Como a linguagem não possui tipo booleano explícito, os operadores relacionais e lógicos geram resultados inteiros. O valor 0 representa falso e qualquer valor diferente de 0 representa verdadeiro.
11. **Uso de regras auxiliares `marca_if` e `marca_while`.** A geração de IR para `if` e `while` exige a emissão de desvios e rótulos antes da redução completa do comando. Para resolver isso com pouca alteração na gramática, foram criadas regras auxiliares que atuam como marcadores de geração de código.
12. **Impressão da IR apenas quando não há erros.** Decidiu-se não imprimir a análise semântica normal nem a IR quando existem erros sintáticos ou semânticos. Essa decisão evita apresentar código intermediário baseado em um programa inválido.
13. **Geração de código intermediário para conversão implícita de tipos.** Foi adicionada a função `widen` para representar na IR a conversão implícita de `int` para `float`. Essa decisão complementa a regra semântica de promoção de tipos, pois, quando uma expressão aritmética com soma, subtração, multiplicação ou divisão resulta em `float`, os operandos inteiros precisam ser convertidos antes da operação principal. Por exemplo, em uma operação entre `int` e `float`, pode ser gerada uma

instrução como `t1 = (float) $x`, e depois a operação final usa esse temporário já convertido.

6 Dificuldades Encontradas

Durante o desenvolvimento da análise semântica e da geração de código intermediário, algumas dificuldades se destacaram.

- **Separação entre erro sintático e erro semântico.** Foi necessário definir que a análise semântica não deveria ser considerada confiável quando houvesse erros sintáticos. Por isso, a saída semântica e a IR são bloqueadas caso o parser encontre erros.
- **Controle de escopos em blocos e funções.** A abertura e o fechamento de escopos exigiram atenção para que variáveis locais, parâmetros e variáveis externas fossem encontrados corretamente sem permitir redeclarações indevidas no mesmo ambiente.
- **Validação de funções.** A parte de funções exigiu maior cuidado porque envolve múltiplos contextos: declaração da função, lista de parâmetros, chamada, argumentos e retorno. A separação entre `funcaoDeclaracaoAtual` e `funcaoChamadaAtual` ajudou a reduzir essa complexidade.
- **Geração de IR em comandos de controle.** A geração de código para `if`, `if/else` e `while` não podia ser feita somente ao final das regras, pois os desvios precisam aparecer antes do corpo dos comandos. A solução foi criar regras auxiliares de marcação.
- **Propagação do endereço das expressões.** Foi necessário garantir que todas as expressões preenchessem corretamente o campo `addr`. Caso esse campo ficasse vazio, a IR poderia gerar instruções incompletas, como temporários sem operandos.
- **Chamada de função com argumentos.** A geração de `param` e `call` exigiu controlar a quantidade de argumentos durante a análise da lista de argumentos. Para isso, foi adicionado o campo `qtdArgumentos` à estrutura de função.

7 Arquivo de Teste e Código Intermediário Gerado

7.1 Arquivo de Entrada

O programa abaixo foi utilizado para testar a análise semântica e a geração de código intermediário. Ele contém declarações de funções, chamadas, atribuições, expressões

aritméticas, relacionais e lógicas, comandos `read` e `print`, estrutura `if/else` e laço `while`.

```
1 int $soma(int $a, int $b) {
2   return $a + $b;
3 }
4 int $maior(int $a, int $b) {
5   if ($a > $b)
6     return $a;
7   else
8     return $b;
9 }
10 int $x = 0;
11 int $y = 3;
12 int $z;
13 float $f = 2.5;
14 read($x);
15 $z = $soma($x, $y);
16 print($z);
17 if (($z > 10) && ($x != 0)) {
18   print("maior");
19 } else {
20   print("menor");
21 }
22 while ($x < 5) {
23   $x = $x + 1;
24   $z = $z + $x;
25 }
26 $z = $z % 2;
27 $z = -$z;
28 $z = !($z == 0);
29 $f = $f + $x * 2;
30 $z = $maior($z, $x);
31 print($x);
32 print($z);
33 print($f);
```

Listagem 8: Arquivo de teste utilizado

7.2 Código Intermediário Gerado

A saída gerada pelo compilador para o código intermediário foi:

```

1 func $soma:
2   t1 = $a + $b
3   return t1
4 end func $soma
5 func $maior:
6   t2 = $a > $b
7   ifFalse t2 goto L1
8   return $a
9   goto L2
10  L1:
11   return $b
12  L2:
13 end func $maior
14 $x = 0
15 $y = 3
16 $f = 2.500000
17 read $x
18 param $x
19 param $y
20 t3 = call $soma, 2
21 $z = t3
22 print $z
23 t4 = $z > 10
24 t5 = $x != 0
25 ifFalse t4 goto L3
26 ifFalse t5 goto L3
27 t6 = 1
28 goto L4
29 L3:
30 t6 = 0
31 L4:
32 ifFalse t6 goto L5
33 print "maior"
34 goto L6
35 L5:
36 print "menor"
37 L6:
38 L7:
39 t7 = $x < 5
40 ifFalse t7 goto L8
41 t8 = $x + 1

```

```

42 $x = t8
43 t9 = $z + $x
44 $z = t9
45 goto L7
46 L8:
47 t10 = $z % 2
48 $z = t10
49 t11 = - $z
50 $z = t11
51 t12 = $z == 0
52 ifFalse t12 goto L9
53 t13 = 0
54 goto L10
55 L9:
56 t13 = 1
57 L10:
58 $z = t13
59 t14 = $x * 2
60 t15 = (float) t14
61 t16 = $f + t15
62 $f = t16
63 param $z
64 param $x
65 t17 = call $maior, 2
66 $z = t17
67 print $x
68 print $z
69 print $f

```

Listagem 9: Código intermediário gerado

Observa-se que a IR gerada contempla temporários, rótulos, desvios condicionais, desvios incondicionais, chamadas de função, passagem de parâmetros, retornos, entrada, saída e expressões. O comando `goto L2` após `return $a` é código inalcançável em tempo de execução, mas não representa erro nesta etapa, pois o projeto não implementa otimizações.

8 Conclusão

A terceira etapa do trabalho permitiu completar as principais fases do front-end do compilador da minilinguagem didática. A análise semântica implementada tornou possível validar o uso de identificadores, tipos, escopos, funções, argumentos e retornos, garantindo

que programas sintaticamente corretos também respeitem as regras semânticas definidas para a linguagem.

A geração de código intermediário consolidou a ligação entre a análise semântica e uma representação mais próxima da execução do programa. Com o uso de temporários, rótulos e instruções de três endereços, foi possível representar expressões, atribuições, comandos de entrada e saída, estruturas de controle e funções de maneira simples e legível.

As decisões de projeto priorizaram uma implementação didática, compatível com o escopo da disciplina e sem a complexidade de um compilador profissional. Dessa forma, não foram implementadas otimizações, DAGs, blocos básicos, grafo de fluxo ou código objeto. Mesmo assim, o compilador produzido atende aos objetivos da etapa, realizando análise semântica e gerando uma IR completa para as principais construções da linguagem.

9 Referências

1. AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Pearson/Addison-Wesley, 2006.
2. TERRA, Ricardo. *Compiladores*. Apostila/slides da disciplina GCC130, Universidade Federal de Lavras (UFLA), 2026.
3. OPENAI. *ChatGPT*. Ferramenta de inteligência artificial utilizada como apoio para revisão textual, organização do relatório e esclarecimentos conceituais. Acesso em: 2026.